

## Personal info

|                |                                |
|----------------|--------------------------------|
| First name     | Sifiso Victor                  |
| Last name      | Masemola                       |
| Display name   | Sifiso Victor Masemola         |
| Email          | sifisovictormasemola@gmail.com |
| Phone          | +27791226371                   |
| Student number | 2805001779                     |

### **Bright learn SQL JOINS**

**Introduction:** This exercise is regarding the inquiry of the following tables. Users, Subscription, viewing session, Plans and shows, where we use INNER, LEFT and FULL JOINS to summarise the outcomes.

**Question 1:** Use Inner join to return only rows where the join condition is true on BOTH sides. Unmatched rows are dropped. Expected rows are User\_id, Username, Subscription\_id, Start\_date.

**Table 1 • users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 3 • subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

## SQL Syntax

**SELECT** User\_id, Username, Subscription\_id, Start\_date

**FROM** Users **AS** A

**INNER JOIN** Subscription **AS** B

**ON** A.User\_id = B.User\_id;

### Summarised table

| User_id | Username | Subscription_id | Start_date |
|---------|----------|-----------------|------------|
| 1       | Nomvula  | 501             | 2026-01-15 |
| 2       | David    | 502             | 2026-02-01 |
| 3       | Anele    | 505             | 2026-04-05 |
| 1       | Nomvula  | 503             | 2026-03-10 |

**QUESTION 2:** Show every subscription with its matching plan name and monthly price.  
**TABLES TO USE** subscriptions · plans  
**EXPECTED COLUMNS** subscription\_id, user\_id, plan\_name, monthly\_price

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

**Table 2 · plans**

| plan_id | plan_name | monthly_price |
|---------|-----------|---------------|
| 10      | Basic     | 79            |
| 11      | Standard  | 129           |
| 12      | Premium   | 199           |
| 13      | Family    | 249           |
| 14      | Mobile    | 59            |

## SQL Syntax

**SELECT** Subscription\_id, User\_id, plan\_name, Monthly\_price

**FROM** Subscription **AS** A

**INNER JOIN** Plans **AS** B

**ON** A.Plan\_id = B.Plan\_id;

## Summarized table

| Subscription_id | Plan_id | Plan_name | Monthly_price |
|-----------------|---------|-----------|---------------|
| 501             | 10      | Basic     | 79            |
| 502             | 11      | Standard  | 129           |
| 503             | 12      | Premium   | 199           |
| 505             | 13      | Family    | 249           |
| 504             | 11      | Standard  | 129           |

**QUESTION 03:** Show every viewing session that has a matching show. Include the show title and genre. **TABLES** `viewing_sessions` · `shows` **EXPECTED COLUMNS** `session_id`, `user_id`, `show_title`, `genre`, `watch_minutes`

**Table 5 · viewing\_sessions**

| session_id | user_id | show_id | watch_minutes |
|------------|---------|---------|---------------|
| 901        | 1       | 701     | 45            |
| 902        | 2       | 703     | 30            |
| 903        | 1       | 702     | 60            |
| 904        | 7       | 701     | 20            |
| 905        | 3       | 705     | 90            |

**Table 4 · shows**

| show_id | show_title  | genre  |
|---------|-------------|--------|
| 701     | Comedy Hour | Comedy |



|     |             |             |
|-----|-------------|-------------|
| 702 | Crime Time  | Drama       |
| 703 | Tech Tales  | Documentary |
| 704 | Cooking Lab | Lifestyle   |
| 706 | Wild Earth  | Documentary |

```

SELECT session_id, User_id, show_title, genre, watch_minutes, show_id
FROM viewing_sessions AS A
INNER JOIN shows AS B
ON A.Show_id = B.Show_id;

```

**Summarised table**

| Session_id | User_id | Show_title  | Genre         | Watch minutes | Show_id |
|------------|---------|-------------|---------------|---------------|---------|
| 901        | 1       | Comedy hour | Comedy        | 45            | 701     |
| 903        | 1       | Crime time  | Drama         | 60            | 702     |
| 902        | 2       | Tech Tales  | Documentation | 30            | 703     |
| 904        | 7       | Comedy hour | Comedy        | 45            | 701     |

**QUESTION 04:** Show every viewing session with the user who watched it. Only show sessions with a matching user. **TABLES TO USE** user · viewing\_sessions **EXPECTED COLUMNS** user\_name, country, session\_id, show\_id, watch\_minutes

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 5 · viewing\_sessions**

| session_id | user_id | show_id | watch_minutes |
|------------|---------|---------|---------------|
| 901        | 1       | 701     | 45            |
| 902        | 2       | 703     | 30            |
| 903        | 1       | 702     | 60            |
| 904        | 7       | 701     | 20            |
| 905        | 3       | 705     | 90            |

**SQL Syntax**

```

SELECT User_name, Country, Session_id, show_id, watch_minutes, user_id
FROM Users AS A
INNER JOIN viewing_sessions AS B
ON A.User_id = B.User_id;

```

**Summarised table**

| User_name | Country      | Session_id | Show_id | Watch_min | User_id |
|-----------|--------------|------------|---------|-----------|---------|
| Nomvula   | Johannesburg | 901        | 701     | 45        | 1       |
| David     | Cape Town    | 902        | 703     | 30        | 2       |
| Anele     | Durban       | 905        | 705     | 60        | 3       |
| Nomvula   | Johannesburg | 903        | 702     | 60        | 1       |

**QUESTION 05:** Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan. **TABLES TO USE** users · subscriptions · plans **EXPECTED COLUMNS** user\_name, country, plan\_name, monthly\_price, start\_date

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

**Table 2 • plans**

| plan_id | plan_name | monthly_price |
|---------|-----------|---------------|
| 10      | Basic     | 79            |
| 11      | Standard  | 129           |
| 12      | Premium   | 199           |
| 13      | Family    | 249           |
| 14      | Mobile    | 59            |

```

SELECT user_name, country, plan_name, monthly_price, start_date, user_id, plan_id
FROM users AS A
INNER JOIN Subscription ON Users.User_id
INNER JOIN plans ON Subscription.plan_id;

```

**OR**

```

SELECT user_name, country, plan_name, monthly_price, start_date, user_id, plan_id
FROM users AS A
INNER JOIN subscription AS B
INNER JOIN Plans AS C
ON A.user_id = B.user_id
ON B.Plan_id = C.Plan_id;

```

**Summarised table**

| User_id | Plan_id | Username | Country      | Plan_name | Monthly price | Start date |
|---------|---------|----------|--------------|-----------|---------------|------------|
| 1       | 10      | Nomvula  | Johannesburg | Basic     | 79            | 2026-01-15 |
| 2       | 11      | David    | Cape Town    | Standard  | 129           | 2026-02-01 |
| 3       | 13      | Anele    | Durban       | Family    | 249           | 2026-04-05 |
| 1       | 12      | Nomvula  | Johannesburg | Premium   | 199           | 2026-03-10 |

The following set of questions include **LEFT JOIN** to summarise the tables given. Part B — LEFT JOIN These five questions use **LEFT JOIN** — every row from the left-hand table is kept, even when it has no match on the right. Watch where **NULL** values appear in your result.

**QUESTION 06:** Show every user and any subscriptions they have. Users without subscriptions must still appear. TABLES TO USE users · subscriptions EXPECTED COLUMNS user\_id, user\_name, subscription\_id, start\_date

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

**SELECT** User\_id, Username, Country, Subscription\_id, Start\_date

**FROM** users **AS** A

**LEFT JOIN** subscription **AS** B

**ON** Users.User\_id = Subscription.User\_id;

**Summarised table**

| User_id | username | country        | Subscription_id | Start_date |
|---------|----------|----------------|-----------------|------------|
| 1       | Nomvula  | Johannesburg   | 501             | 2026-01-15 |
| 2       | David    | Cape Town      | 502             | 2026-02-01 |
| 3       | Anele    | Durban         | 505             | 2026-04-05 |
| 4       | Kabelo   | Pretoria       | NULL            | NULL       |
| 5       | Lerato   | Port Elizabeth | NULL            | NULL       |

**QUESTION 07:** Show every plan and the subscriptions on it. Plans with no subscribers must still appear. TABLES TO USE plans · subscriptions EXPECTED COLUMNS plan\_id, plan\_name, subscription\_id, user\_id

**Table 2 · plans**

| plan_id | plan_name | monthly_price |
|---------|-----------|---------------|
| 10      | Basic     | 79            |
| 11      | Standard  | 129           |
| 12      | Premium   | 199           |
| 13      | Family    | 249           |
| 14      | Mobile    | 59            |

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

**SELECT** plan\_id, plan\_name, subscription\_id, user\_id,

**FROM** plans **AS** A

**LEFT JOIN** Subscription **AS** B

**ON** plans.plan\_id = subscription.plan\_id;

**Summarised table**

| Plan_id | Plan_name | Subscription_id | User_id |
|---------|-----------|-----------------|---------|
| 10      | Basic     | 501             | 1       |
| 11      | Standard  | 502             | 2       |
| 12      | Premium   | 503             | 1       |
| 13      | Family    | 505             | 3       |
| 14      | Mobile    | NULL            | NULL    |

**QUESTION 08:** Show every show and any viewing sessions on it. Shows that were never watched must still appear. **TABLES TO USE** shows · viewing\_sessions **EXPECTED COLUMNS** show\_id, show\_title, session\_id, watch\_minutes



**Table 4 · shows**

| show_id | show_title  | genre  |
|---------|-------------|--------|
| 701     | Comedy Hour | Comedy |

BrightLearn · SQL Joins Exercise

Page 5



BrightLearn

SQL JOINS  
EXERCISE

|     |             |             |
|-----|-------------|-------------|
| 702 | Crime Time  | Drama       |
| 703 | Tech Tales  | Documentary |
| 704 | Cooking Lab | Lifestyle   |
| 706 | Wild Earth  | Documentary |

**Table 5 · viewing\_sessions**

| session_id | user_id | show_id | watch_minutes |
|------------|---------|---------|---------------|
| 901        | 1       | 701     | 45            |
| 902        | 2       | 703     | 30            |
| 903        | 1       | 702     | 60            |
| 904        | 7       | 701     | 20            |
| 905        | 3       | 705     | 90            |

**SELECT** show\_id, show\_title, session\_id, watch\_minutes

**FROM** shows **AS** A

**LEFT JOIN** viewing session **AS** B

**ON** shows.show\_id = viewing session.show\_id;

**Summarised table**

| Show_id | Show_title  | Genre       | Session_id | Watching_minutes |
|---------|-------------|-------------|------------|------------------|
| 701     | Comedy hour | Comedy      | 901        | 45               |
| 702     | Crime time  | Drama       | 903        | 60               |
| 703     | Tech Tales  | Documentary | 902        | 30               |
| 704     | Cooking lab | Lifestyle   | NULL       | NULL             |
| 706     | Wild lab    | Documentary | NULL       | NULL             |
| 705     | NULL        | NULL        | 905        | 60               |

**QUESTION 09:** Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details). TABLES TO USE viewing\_sessions · users EXPECTED COLUMNS session\_id, show\_id, watch\_minutes, user\_id, user\_name

**Table 5 · viewing\_sessions**

| session_id | user_id | show_id | watch_minutes |
|------------|---------|---------|---------------|
| 901        | 1       | 701     | 45            |
| 902        | 2       | 703     | 30            |
| 903        | 1       | 702     | 60            |
| 904        | 7       | 701     | 20            |
| 905        | 3       | 705     | 90            |

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**SELECT** session\_id, show\_id, user\_id, watch\_minutes, user\_name

**FROM** viewing session **AS** A

**LEFT JOIN** users **AS** B

**ON** viewing session.user\_id = users.user\_id;

**Summarised table**

| Session_id | User_id | Show_id | Watch_minutes | User_name |
|------------|---------|---------|---------------|-----------|
| 901        | 1       | 701     | 45            | Nomvula   |
| 902        | 2       | 703     | 30            | David     |
| 903        | 1       | 702     | 60            | Nomvula   |
| 904        | 7       | 701     | 20            | NULL      |
| 905        | 3       | 705     | 60            | Anele     |
| NULL       | 5       | NULL    | NULL          | Lerato    |

**QUESTION 10** Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear. TABLES TO USE users · subscriptions · plans EXPECTED COLUMNS user\_name, country, plan\_name, monthly\_price

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 2 · plans**

| plan_id | plan_name | monthly_price |
|---------|-----------|---------------|
| 10      | Basic     | 79            |
| 11      | Standard  | 129           |
| 12      | Premium   | 199           |
| 13      | Family    | 249           |
| 14      | Mobile    | 59            |

**SELECT** user\_name, country, plan\_name, monthly\_price, user\_id, plan\_id

**FROM** users **AS** A

**LEFT JOIN** Subscription **AS** B

**LEFT JOIN** plans **AS** C

**ON** users.user\_id = subscription.user\_id

**ON** plans.plan\_id = subscription.plan\_id;

**Summarised table**

| User_id | Plan_id | User_name | country      | Plan_name | Monthly_price |
|---------|---------|-----------|--------------|-----------|---------------|
| 1       | 10      | Nomvula   | Johannesburg | Basic     | 79            |
| 2       | 11      | David     | Cape Town    | Standard  | 129           |

|   |      |         |                |          |      |
|---|------|---------|----------------|----------|------|
| 3 | 13   | Anele   | Durban         | family   | 249  |
| 4 | NULL | Kabelo  | Pretoria       | NULL     | NULL |
| 5 | NULL | Lerato  | Port Elizabeth | NULL     | NULL |
| 1 | 12   | Nomvula | Johannesburg   | Premium  | 199  |
| 6 | 11   | NULL    | NULL           | Standard | 129  |

**Part C — FULL OUTER JOIN** The final five questions use **FULL OUTER JOIN** — every row from both tables, matched where possible and **NULL** elsewhere. This is the join that reveals everything missing on both sides at once.

**QUESTION 11:** Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist. TABLES TO USE users · subscriptions EXPECTED COLUMNS user\_id, user\_name, subscription\_id, start\_date

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

```

SELECT user_id,user_name,subscription__id,country,,start_date,plan_id
FROM users AS A
FULL OUTER JOIN subscriptions AS B
ON users.user_id = subscriptions.user_id;

```

### Summarised Table

| User_id | username | country        | Subscription_id | Plan_id | Start_date |
|---------|----------|----------------|-----------------|---------|------------|
| 1       | Nomvula  | Johannesburg   | 501             | 10      | 2026-01-15 |
| 2       | David    | Cape Town      | 502             | 11      | 2026-02-01 |
| 3       | Anele    | Durban         | 505             | 13      | 2026-04-05 |
| 4       | Kabelo   | Pretoria       | NULL            | NULL    | NULL       |
| 5       | Lerato   | Port Elizabeth | NULL            | NULL    | NULL       |
| 6       | NULL     | NULL           | 504             | 11      | 2026-03-20 |

**QUESTION 12** Show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist. TABLES TO USE plans · subscriptions EXPECTED COLUMNS plan\_id, plan\_name, subscription\_id, user\_id

**Table 2 · plans**

| plan_id | plan_name | monthly_price |
|---------|-----------|---------------|
| 10      | Basic     | 79            |
| 11      | Standard  | 129           |
| 12      | Premium   | 199           |
| 13      | Family    | 249           |
| 14      | Mobile    | 59            |

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |

### SELECT

plan\_id,plan\_name,subscriptions\_id,monthly\_plan,user\_id,plan\_name,start\_date

**FROM** plans **AS** A

**FULL OUTER JOIN** subscription **AS** B

**ON** plans.plan\_id = subscription.plan\_id

## Summarised table

| Plan_id | Plan_name | Monthly_plan | Subscription_id | User_id | Start_date |
|---------|-----------|--------------|-----------------|---------|------------|
| 10      | Basic     | 79           | 501             | 1       | 2026-01-15 |
| 11      | Standard  | 129          | 502             | 2       | 2026-02-01 |
| 12      | Premium   | 199          | 503             | 1       | 2026-03-10 |
| 13      | Family    | 249          | 505             | 3       | 2026-04-05 |
| 14      | Mobile    | 59           | NULL            | NULL    | NULL       |
| 11      | Standard  | 129          | 504             | 6       | 2026-03-26 |

**QUESTION 13** Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist. TABLE TO USE shows · viewing\_sessions EXPECTED COLUMNS show\_id, show\_title, session\_id, watch\_minutes

Table 4 · shows

| show_id | show_title  | genre  |
|---------|-------------|--------|
| 701     | Comedy Hour | Comedy |

BrightLearn · SQL Joins Exercise

Page 5



BrightLearn

SQL JOINS  
EXERCISE

|     |             |             |
|-----|-------------|-------------|
| 702 | Crime Time  | Drama       |
| 703 | Tech Tales  | Documentary |
| 704 | Cooking Lab | Lifestyle   |
| 706 | Wild Earth  | Documentary |

Table 5 · viewing\_sessions

| session_id | user_id | show_id | watch_minutes |
|------------|---------|---------|---------------|
| 901        | 1       | 701     | 45            |
| 902        | 2       | 703     | 30            |
| 903        | 1       | 702     | 60            |
| 904        | 7       | 701     | 20            |
| 905        | 3       | 705     | 90            |

**SELECT** show\_id,show\_title,genre,session\_id,user\_id,watch\_minutes

**FROM** shows **AS** A

**FULL OUTER JOIN** viewing session **AS** B

**ON** shows.show\_id = viewing session.show\_id;

**Summarised table**

| Show_id | Show_title  | genre       | Session_id | User_id | Watch_minutes |
|---------|-------------|-------------|------------|---------|---------------|
| 701     | Comedy hour | Comedy      | 901        | 1       | 45            |
| 702     | Crime time  | Drama       | 903        | 1       | 60            |
| 703     | Tech tale   | Documentary | 902        | 2       | 30            |
| 704     | Cooking lab | Lifestyle   | NULL       | NULL    | NULL          |
| 705     | NULL        | NULL        | 905        | 3       | 60            |
| 706     | Wild earth  | Documentary | NULL       | NULL    | NULL          |
| 701     | Comedy hour | comedy      | 904        | 7       | 20            |

**QUESTION 14** Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist. TABLES TO USE users · viewing\_sessions EXPECTED COLUMNS user\_id, user\_name, session\_id, show\_id, watch\_minutes

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 5 · viewing\_sessions**

| session_id | user_id | show_id | watch_minutes |
|------------|---------|---------|---------------|
| 901        | 1       | 701     | 45            |
| 902        | 2       | 703     | 30            |
| 903        | 1       | 702     | 60            |
| 904        | 7       | 701     | 20            |
| 905        | 3       | 705     | 90            |

**SELECT** user\_id,user\_name,country,session\_id,show\_id,watch\_minutes

**FROM** users **AS** A

**FULL OUTER JOIN** viewing session **AS** B

**ON** users.user\_id = viewing session.user\_id;

**Summarised table**

| User_id | username | country        | Session_id | Show_id | Watch_minutes |
|---------|----------|----------------|------------|---------|---------------|
| 1       | Nomvula  | Johannesburg   | 901        | 701     | 45            |
| 2       | David    | Cape Town      | 902        | 703     | 30            |
| 3       | Anele    | Durban         | 905        | 705     | 66            |
| 1       | Nomvula  | Johannesburg   | 903        | 702     | 60            |
| 4       | Kabelo   | Pretoria       | NULL       | NULL    | NULL          |
| 5       | Lerato   | Port Elizabeth | NULL       | NULL    | NULL          |
| 7       | NULL     | NULL           | 904        | 701     | 20            |

**QUESTION 15:** Show every user, every subscription, and every plan in one query — using FULL OUTER JOIN throughout. This is the hardest question — get all gaps visible at once. **TABLES TO USE** users · subscriptions · plans **EXPECTED COLUMNS** user\_id, user\_name, subscription\_id, plan\_id, plan\_name

**Table 1 · users**

| user_id | user_name | country        |
|---------|-----------|----------------|
| 1       | Nomvula   | Johannesburg   |
| 2       | David     | Cape Town      |
| 3       | Anele     | Durban         |
| 4       | Kabelo    | Pretoria       |
| 5       | Lerato    | Port Elizabeth |

**Table 3 · subscriptions**

| subscription_id | user_id | plan_id | start_date |
|-----------------|---------|---------|------------|
| 501             | 1       | 10      | 2026-01-15 |
| 502             | 2       | 11      | 2026-02-01 |
| 503             | 1       | 12      | 2026-03-10 |
| 504             | 6       | 11      | 2026-03-20 |
| 505             | 3       | 13      | 2026-04-05 |



**Table 2 · plans**

| plan_id | plan_name | monthly_price |
|---------|-----------|---------------|
| 10      | Basic     | 79            |
| 11      | Standard  | 129           |
| 12      | Premium   | 199           |
| 13      | Family    | 249           |
| 14      | Mobile    | 59            |

## SELECT

user\_id,plan\_id,username,country,plan\_name,subscription\_id,start\_date,monthly\_plan

**FROM** users **AS** A

**FULL OUTER JOIN** subscription **AS** B

**FULL OUTER JOIN** plans **AS** C

**ON** users.user\_id = subscriptions.user\_id

**ON** subscriptions.plan\_id = plans.plan\_id;

## Summarised table

| User id | Plan id | Username | country        | Plan name | Subscription id | Start date | Monthly plan |
|---------|---------|----------|----------------|-----------|-----------------|------------|--------------|
| 1       | 10      | Nomvula  | Johannesburg   | Basic     | 501             | 2026-01-13 | 79           |
| 2       | 11      | David    | Cape Town      | Standard  | 502             | 2026-02-01 | 129          |
| 3       | 13      | Anele    | Durban         | Family    | 505             | 2026-04-05 | 249          |
| 4       | NULL    | Kabelo   | Pretoria       | NULL      | NULL            | NULL       | NULL         |
| 5       | NULL    | Lerato   | Port Elizabeth | NULL      | NULL            | NULL       | NULL         |
| 6       | 11      | David    | Cape Town      | Standard  | 504             | 2026-03-26 | 129          |
| 1       | 12      | Nomvula  | Johannesburg   | Premium   | 503             | 2026-03-10 | 199          |
| NULL    | 14      | NULL     | NULL           | Mobile    | NULL            | NULL       | 59           |

**Bonus Challenge** Now the conceptual test. Without writing any more SQL, answer these five questions from looking at the tables — they prove you really understand how the joins work and what they reveal about your data.

**B O N U S 0 1** Which users have not subscribed to any plan?

- Kabelo (user\_id 4) and Lerato (user\_id 5)

**B O N U S 0 2** Which subscriptions reference users that do not exist in the users table?

- Subscription\_id (Subscription table)

**BONUS 03** Which shows have never been watched?

- Wild earth (documentary)

**BONUS 04** Which viewing sessions reference shows that do not exist?

- Cooking Lab (lifestyle)

**BONUS 05** Which plans have no subscribers?

- Mobile (Plan\_id 14)

**Conclusion:** the findings during this task was that some users have not subscribed and using joining three columns at once using SQL syntax was a bit challenging, but I managed to analyse using the method of writing down before coding.